Sorting as Gradient Flow on the Permutohedron

Jonathan Robert Landers
jonathan.robert.landers@gmail.com

Abstract

We investigate how sorting algorithms navigate the complexity of permutation space. The main contribution is a geometric and dynamical model of comparison sorting on the permutohedron. It organizes three complementary views of order types: adjacent-swap paths along the 1-skeleton, comparison half-spaces that refine the feasible order types, and a quadratic potential whose ambient gradient flow contracts toward the fixed sorted vertex $v_s = (1, 2, \ldots, n)$. Comparisons remove informational ambiguity, whereas the flow removes metric distance; the two processes are complementary rather than literally the same dynamics. To support this analysis, we present decision-tree arguments, adjacent-swap paths with $\Theta(n^2)$ behavior, and linear constraints on candidate rank maps. The classical optimal comparison scale $\Theta(n \log n)$ and the continuous contraction time $\Theta(\log n)$ then share a logarithmic scale after comparisons are normalized by n. This geometric perspective connects to broader questions in theoretical computer science, such as whether the existence of structure can explain why certain computational problems permit efficient solutions.

MSC 2020: 68Q25, 52B11, 90C52
Keywords: sorting algorithms, permutohedron, gradient flow, computational complexity

GitHub repository Project website

1 Introduction

Consider the fundamental problem of sorting a list

$$L = [\ell_1, \ell_2, \ldots, \ell_n]. \tag{1}$$

The elements are taken to lie in a totally ordered set so that their relative order is well-defined. Rather than viewing sorting merely as a sequence of symbolic operations, it is useful to regard it as navigation through a large structured space. Each input determines one of $n!$ possible order types, and comparisons locate the correct one. Different computational models then reveal different notions of motion, distance, and progress within that space.

Notation. The key distinction is between an item's original identity and its current position. We therefore keep four related objects separate. The integer i is the original index of an item, and ℓ_i is the value at that index. Since the entries are assumed pairwise distinct, define the original rank map

$$\rho(i) = \mathrm{rank}(\ell_i), \tag{2}$$

where rank 1 is smallest and rank n is largest. At algorithmic time t, let $p_t(k)$ be the original index of the item currently occupying array position k. The rank word plotted on the permutohedron is

$$x_t(k) = \rho(p_t(k)). \tag{3}$$

Thus x_t records the ranks currently sitting in array positions, and initially

$$x_0 = (\rho(1), \rho(2), \ldots, \rho(n)). \tag{4}$$

The rank word is used here as an ex post representation of the input order type. Constructing it from the raw input already requires knowledge of the rank map, so the continuous trajectory is a geometric relaxation rather than an algorithm for discovering the unknown ranks. Sorting moves the rank word x_0 toward $v_s = (1, 2, \ldots, n)$. The sorted index order is a different object. Here $\sigma(k)$ is the original index of the item occupying sorted position k, so $\sigma = \rho^{-1}$, and

$$[\ell_{\sigma(1)}, \ell_{\sigma(2)}, \ldots, \ell_{\sigma(n)}] \tag{5}$$

is sorted. For example, if $L = [3, 1, 2]$, then $\rho = (3, 1, 2)$ but $\sigma = (2, 3, 1)$. These differ in general, even though for $L = [5, 4, 2]$ both may be written as 321.

We write τ for a permutation of the index set $\{1, 2, \ldots, n\}$, acting on L by

$$\tau(L) = [\ \ell_{\tau(1)},\ \ell_{\tau(2)},\ \ldots,\ \ell_{\tau(n)}\ ]. \tag{6}$$

Setting. A brute-force baseline is to enumerate all $n!$ permutations $\tau \in S_n$ and select σ, the permutation that produces the sorted list. This costs $\Theta(n \cdot n!)$, since each candidate ordering requires $O(n)$ time to check. We use standard asymptotic notation as in [1, 2]. Bounds are worst-case unless stated otherwise, randomized costs are denoted by $\mathbb{E}[T(n)]$, and all logarithms are base 2.

Within this framework, established sorting algorithms such as MergeSort, HeapSort, and QuickSort achieve a remarkable improvement, reducing the search from factorial to log-linear scaling. By contrast, local-swap methods such as BubbleSort use only adjacent comparisons and swaps, yielding quadratic $\Theta(n^2)$ worst-case behavior. MergeSort and HeapSort guarantee $O(n \log n)$ worst-case performance, while QuickSort attains the same bound in expectation. Each exploits structure in the input, effectively pruning large regions of permutation space.

This striking efficiency has long inspired curiosity about the connection between structure and computational tractability. Foundational work by Knuth [3], Shannon [4], and others, along with recent combinatorial and geometric perspectives [5, 6, 7], reflects ongoing efforts to understand how structure constrains and organizes computation.

The main innovation presented in this paper is a continuous-time formulation of sorting as an ambient gradient flow on the permutohedron, placed alongside the discrete geometries induced by sorting operations themselves. Adjacent swaps determine a local graph geometry, comparisons determine an information geometry on candidate rank maps, and the Euclidean flow supplies a smooth benchmark for contraction. This framework gives a geometric and dynamical reframing of the classical $\Omega(n \log n)$ lower bound for comparison-based sorting without replacing the decision-tree proof. It organizes the sources of efficiency geometrically: local moves walk through the polytope, while global comparisons carve and organize the feasible space.

We begin with the decision-tree lower bound to set the complexity scale, then reinterpret comparisons as linear constraints acting on the permutohedron, and finally introduce a gradient-flow model that makes geometric contraction explicit.

2 Information-Theoretic Decision-Tree Lower Bound

The lower bound for any comparison-based sorting algorithm is first established by modeling the process as a binary decision tree, whose leaves distinguish the possible strict order types, equivalently the required output permutations.

Lemma 2.1 (Decision-Tree Lower Bound)**.** *Let T be a binary decision tree that correctly sorts n elements using comparisons. Then the height h of T satisfies*

$$h \geq \log(n!). \tag{7}$$

Proof. A deterministic comparison-based algorithm must distinguish the $n!$ possible strict orderings of the input. Two distinct order types cannot terminate at the same leaf, because a leaf prescribes a single output permutation. Consequently, the decision tree must have at least $n!$ leaves. Because any binary tree of height h has at most 2^h leaves, we obtain:

$$2^h \geq n!. \tag{8}$$

Taking the base-2 logarithm of both sides yields:

$$h \geq \log(n!). \tag{9}$$

Using Stirling's approximation,

$$\log(n!) = n \log n - O(n), \tag{10}$$

so that $h = \Omega(n \log n)$. This lower bound is tight for comparison-based sorting algorithms. □

This information-theoretic picture also has an enumerative counterpart. Harris, Kretschmann, and Martínez Mori [6] have shown that the total number of comparisons made by QuickSort over all $n!$ input orderings coincides with the number of parking-preference lists admitting exactly $n - 1$ lucky cars. This identity further underscores how comparison structure governs the complexity of sorting.

Figure 1 illustrates the decision-tree framework through a tree for sorting three elements, instantiated with the input $L = (\ell_1 = 5, \ell_2 = 4, \ell_3 = 2)$. The leaves are labeled by candidate sorting permutations of the original indices, and the corresponding output at a leaf is the reordered list. The highlighted root-to-leaf path records the three comparisons performed on this input and terminates at the leaf labeled $\sigma = (3, 2, 1)$, which yields the sorted output $[2, 4, 5]$. Thus the example realizes $\lceil \log(3!) \rceil = 3$, in agreement with the lower bound. Knuth's seminal analysis established this information-theoretic scale for sorting algorithms [3].

The decision-tree model fixes the comparison scale, but it does not explain the geometry of the space being carved. This distinction matters: algorithms restricted to local adjacent comparisons, such as BubbleSort, resolve one inversion at a time and have worst-case cost $\Theta(n^2)$, while arbitrary comparisons can approach the balanced $\Theta(n \log n)$ scale. In the next section we express this contrast on the permutohedron by interpreting comparison outcomes as linear inequalities. Sorting then appears not only as logical branching but as structured contraction of the candidate set until a single rank map remains.

3 Geometric Perspective: The Permutohedron and Half-Space Constraints

The same sorting process now admits two geometric readings on the permutohedron. Adjacent swaps trace paths along its edges and produce the familiar $\Theta(n^2)$ behavior of local sorting algorithms. Comparisons act differently: as linear inequalities, they remove incompatible order types from consideration. Thus one geometry describes local motion, while the other describes information refinement on the way to the optimal $\Theta(n \log n)$ comparison scale.

Definition 3.1 (Permutohedron)**.** Let

$$v_s = (1, 2, \ldots, n) \in \mathbb{R}^n. \tag{11}$$

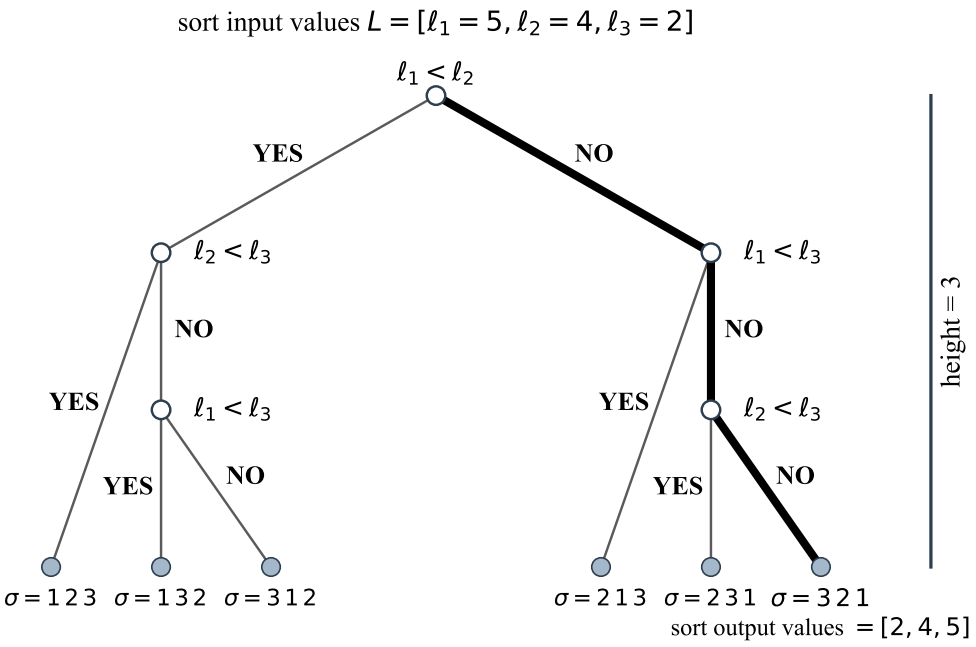

Figure 1: Decision tree for sorting $L = (\ell_1 = 5, \ell_2 = 4, \ell_3 = 2)$. Internal nodes are comparisons and leaves are candidate sorting permutations. The highlighted path identifies $\sigma = (3, 2, 1)$, yielding $[2, 4, 5]$ in three comparisons, matching $\lceil \log(3!) \rceil = 3$. Later permutohedron figures use rank-word coordinates rather than these index-order leaf labels.

The permutohedron $\mathcal{P}_n$ is defined as

$$\mathcal{P}_n = \text{conv}\{\pi(v_s) \mid \pi \in S_n\}. \tag{12}$$

Its vertex set is $\mathcal{V}_n = \{\pi(v_s) \mid \pi \in S_n\}$. In this paper these vertices are read as rank words: the coordinate in position k is the rank currently occupying that position.

Proposition 3.2 (Dimensionality and Affine Containment)**.** *The permutohedron $\mathcal{P}_n$ is an $(n-1)$-dimensional polytope contained in the affine hyperplane*

$$H = \left\{ x \in \mathbb{R}^n \,\middle|\, \sum_{i=1}^{n} x_i = \frac{n(n+1)}{2} \right\}. \tag{13}$$

Proof. For any permutation $\pi \in S_n$,

$$\sum_{i=1}^{n} (\pi(v_s))_i = \sum_{i=1}^{n} i = \frac{n(n+1)}{2}. \tag{14}$$

Thus, every vertex of $\mathcal{P}_n$ lies in the hyperplane H.

To see that these vertices span H, note that for each adjacent transposition $(i, i+1)$,

$$(i, i+1)(v_s) - v_s = \pm(e_{i+1} - e_i), \tag{15}$$

with the sign depending only on the convention for writing the permutation action. The vectors $e_{i+1} - e_i$ for $i = 1, \ldots, n-1$ form a basis of the subspace

$$\left\{ x \in \mathbb{R}^n : \sum_{i=1}^{n} x_i = 0 \right\}. \tag{16}$$

Since adjacent transpositions generate all permutations, it follows that the set $\{\pi(v_s) : \pi \in S_n\}$ affinely spans the entire hyperplane H.

Because H has dimension $n-1$, the permutohedron $\mathcal{P}_n$ is an $(n-1)$-dimensional polytope. □

3.1 Adjacent-Swap Walks and Their Geometric Interpretation

Before introducing the linear-constraint formulation, it is helpful to visualize how rank-word vertices relate to one another on the permutohedron $\mathcal{P}_n$. Because adjacent transpositions generate S_n, they appear naturally as edges of the 1-skeleton (Cayley graph) of $\mathcal{P}_n$. Traversing these edges resolves one inversion at a time. It gives a literal geometric realization of sorting as motion on a polytope and mirrors the stepwise progression in the decision-tree example for the input $L = (\ell_1, \ell_2, \ell_3) = (5, 4, 2)$. The walk diagrams use an inverse labeling of the standard permutohedron so that exchanging neighboring array positions corresponds to traversing an edge. This differs from direct coordinate labeling by permutation inversion.

Figure 2 depicts this adjacent-swap path for $n = 3$ in two complementary forms: a flattened 2-dimensional schematic and a full 3-dimensional embedding of $\mathcal{P}_3$. In both views, the highlighted trajectory connects the reverse rank word $(3, 2, 1)$ to the sorted rank word $(1, 2, 3)$ by successive neighbor exchanges. These local moves provide a geometric model for the $\Theta(n^2)$ behavior of adjacent-swap sorting algorithms. The low-dimensional example is used only because $\mathcal{P}_3$ can be drawn directly; the definitions, constraints, and flow estimates below are stated for arbitrary n.

An edge path records how the current arrangement changes. A comparison history answers a different question: which original rank maps remain possible? When its outcomes are recorded as inequalities between candidate ranks of the original items, they define half-spaces whose intersection with $\mathcal{V}_n$ narrows the search space. Proposition 3.3 connects this refinement to the information-theoretic scale of $\Omega(n \log n)$ comparisons.

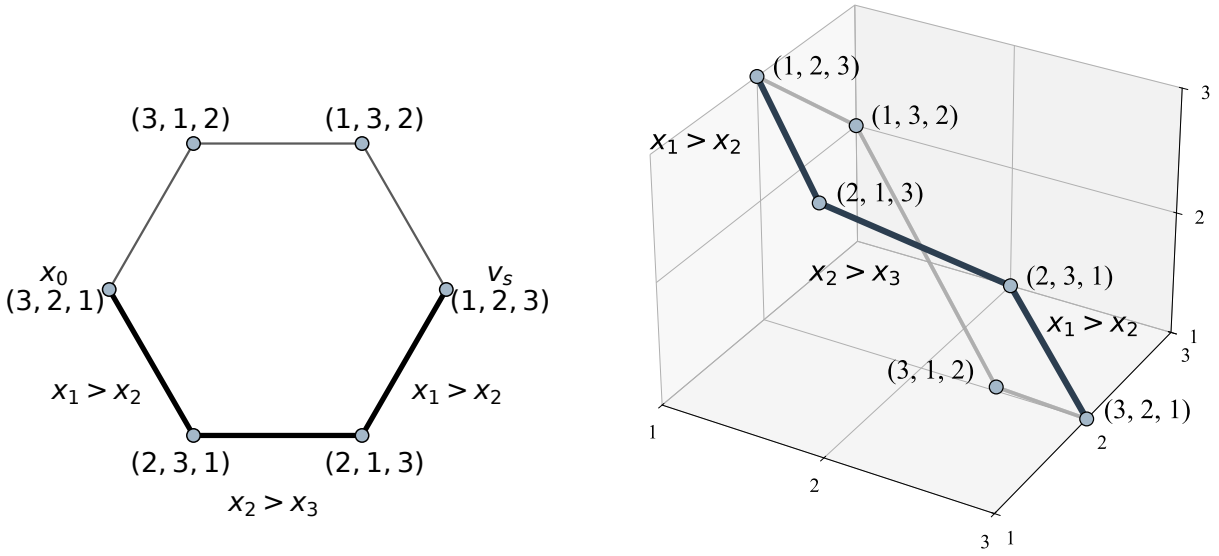

Figure 2: Adjacent-position-swap motion on an inverse-labeled permutohedron $\mathcal{P}_3$. **Left:** A 2D schematic of local traversal along the 1-skeleton. **Right:** The same array-rank path $(3, 2, 1) \to (2, 3, 1) \to (2, 1, 3) \to (1, 2, 3)$ in a 3D embedding. Each step exchanges neighboring positions, illustrating the local $\Theta(n^2)$ geometry of adjacent-swap sorting.

3.2 Linear Constraints and Information Refinement

Comparisons at different stages must be placed in a common coordinate frame. A comparison is made between two current positions, but the information it reveals concerns the two original items occupying those positions. To combine all comparison outcomes into one fixed feasible set, the inequalities must therefore be recorded in original-item coordinates.

Let

$$r = (r_1, \ldots, r_n) \in \mathcal{V}_n \tag{17}$$

be a candidate original-index rank map, where r_i is the candidate rank of the original item ℓ_i. If at time t a comparison between positions a and b has the "less than" outcome, it records

$$r_{p_t(a)} < r_{p_t(b)}. \tag{18}$$

In plain language, if position a currently contains item i and position b contains item j, the comparison records $r_i < r_j$.

For a fixed comparison history C, let $\lhd_j$ be $<$ or $>$ according to the outcome of comparison j, made at time t_j between positions a_j and b_j. Its feasible candidate rank maps are

$$\mathcal{R}_n(C) = \left\{ r \in \mathcal{V}_n : r_{p_{t_j}(a_j)} \lhd_j r_{p_{t_j}(b_j)} \text{ for every comparison } j \right\}. \tag{19}$$

Proposition 3.3 (Comparison Constraints and Candidate Rank Maps)**.** *For an input with original rank map ρ, a complete comparison history C at a decision-tree leaf identifies that map:*

$$\mathcal{R}_n(C) = \{\rho\}. \tag{20}$$

Consequently, a correct comparison decision tree has worst-case depth at least $\log(n!) = \Omega(n \log n)$, while optimal comparison sorts supply the matching $O(n \log n)$ upper bound. If $\sigma = \rho^{-1}$ is the output permutation, then applying it converts the identified rank map to the sorted rank word:

$$\big(\rho(\sigma(1)), \ldots, \rho(\sigma(n))\big) = (1, \ldots, n) = v_s. \tag{21}$$

Proof. Along a fixed decision-tree path, the positions compared and the original items occupying them are determined by the preceding outcomes and operations. A candidate rank map r follows this path exactly when it satisfies every recorded inequality, so $\mathcal{R}_n(C)$ is precisely the set of order types compatible with the history.

At a leaf, the algorithm has fixed one output permutation. Correctness requires that this permutation sort every candidate reaching that leaf, but an original-index rank map r is sorted by the unique permutation r^{-1}. Thus only one candidate rank map can reach a correct leaf. Since the true map ρ follows the recorded path, $\mathcal{R}_n(C) = \{\rho\}$, and applying $\sigma = \rho^{-1}$ gives the displayed sorted rank word.

In the decision-tree model, each comparison has only two possible outcomes. Thus, at best, a comparison can distinguish between two classes of remaining candidate rank words. For the purpose of proving a lower bound, we may therefore use the optimally balanced decision-tree ideal, in which each comparison splits the remaining feasible candidates as evenly as possible. This is the most optimistic binary refinement available to any comparison-based algorithm: not every comparison produces an exactly balanced cut, and arbitrary comparison hyperplanes need not halve either the vertices or the volume of the permutohedron.

In this ideal halving model, after k comparisons,

$$\frac{n!}{2^k} \le 1 \quad \Rightarrow \quad k \ge \log(n!) = \Omega(n \log n). \tag{22}$$

Classical algorithms achieve $O(n \log n)$ comparisons. Hence their constraints cut down the candidate original-index rank maps to exactly one feasible vertex, supplying the matching upper bound. □

Figure 3 gives a canonical fixed-coordinate illustration of the decision-tree halving heuristic. The inequalities $x_1 < x_2$ and $x_2 < x_3$ isolate the increasing chamber and hence the vertex $(1, 2, 3)$. This view is schematic rather than a literal reconstruction of the adaptive comparison path for the input $[5, 4, 2]$. For a concrete execution, comparison outcomes are first recorded in original-item coordinates and are then converted to sorted-output coordinates. In the optimally balanced decision-tree ideal, the panels represent exponential reduction in the number of feasible permutations; an actual comparison need not split the remaining vertices or polytope volume evenly.

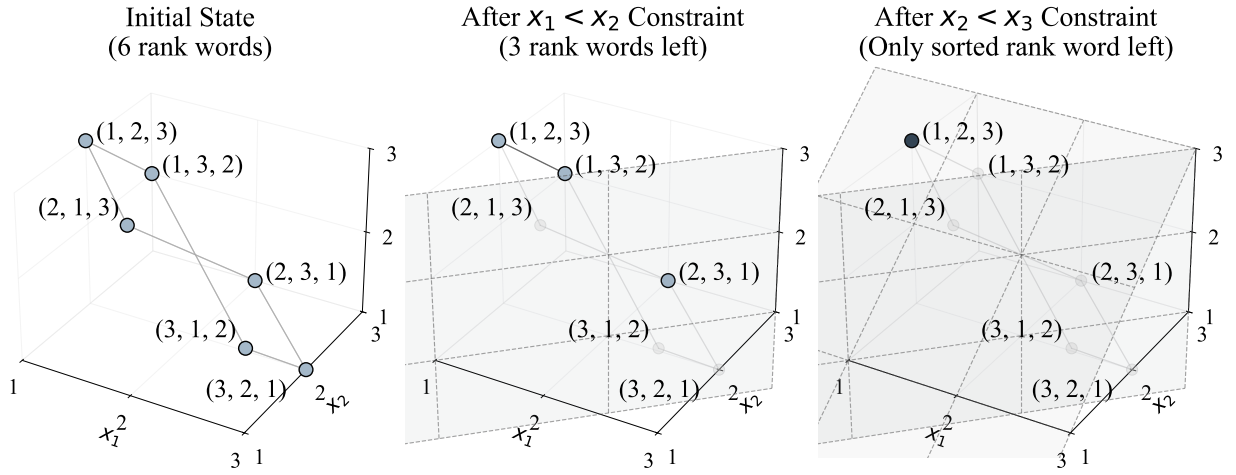

Figure 3: Canonical comparison half-spaces on $\mathcal{P}_3$. The cuts $x_1 < x_2$ and $x_2 < x_3$ isolate the increasing rank word. The picture illustrates chamber refinement rather than the literal adaptive comparison path of a particular input.

Blelloch and Dobson give a related link between structure and complexity. Through a planar representation of permutations and search sequences, they connect comparison sorting to offline binary search trees. Their log-interleave bound offers an information-theoretic lens on sorting cost [5]. While their framework targets BST permutations, our viewpoint treats comparisons as half-space constraints on the permutohedron, emphasizing a general information-refinement mechanism.

A related geometric approach is explored by Lee et al., who study stack-sorting algorithms via subpolytopes of the permutohedron called stack-sorting simplices [8]. These structures capture how the stack-sort process traverses faces and edges of the permutohedron to identify sortable permutations. Their work provides a detailed case study of structured decomposition within a constrained algorithm. In contrast, the present analysis considers general comparison sorts and treats comparisons as half-space constraints that carve through the full permutohedron. This broader perspective builds on such algorithm-specific insights to describe how comparison histories isolate an input order type.

The three viewpoints now on the table use different states and different measures of progress. With adjacent exchanges, this is graph distance on the 1-skeleton of $\mathcal{P}_n$, equivalently the number of inversions to be removed. With arbitrary comparisons, the state is the set of feasible original-index rank maps, and disorder is the information still needed to isolate one of them. Euclidean distance in the standard embedding is a third object.

The central comparison therefore separates two geometrically different processes. First, comparisons eliminate incompatible order types until a single permutation remains. Second, once the target permutation is represented as a vertex, Euclidean gradient flow gives a continuous model of contraction toward it. These mechanisms are not identical: one is informational and the other metric. Their connection is a shared logarithmic scale after comparisons are normalized by n. We now make the metric side of this picture

precise.

4 Continuous Gradient Flow and the Classical $n \log n$ Scale

The metric process begins with a fixed target. The ambient gradient flow makes contraction toward that target in continuous time explicit, while the decision-tree model continues to supply the information scale.

4.1 Setup and Definitions

Let

$$v_s = (1, 2, \ldots, n) \in \mathbb{R}^n \tag{23}$$

denote the vertex of the permutohedron corresponding to the sorted rank word. For the continuous relaxation, measure rank displacement in the standard Euclidean embedding of the permutohedron and define the potential

$$V(x) = \frac{1}{2}\|x - v_s\|^2 = \frac{1}{2}\sum_{i=1}^{n}(x_i - i)^2. \tag{24}$$

This is a Spearman-type squared rank displacement inherited from the ambient embedding. It is not the only sorting metric: adjacent-swap sorting is measured by inversion distance, while comparison sorting is measured by the information remaining in the set of candidate rank maps. The role of V is to provide a smooth benchmark on $\mathcal{P}_n$. Note that $V(v_s) = 0$ and $V(x) > 0$ for $x \neq v_s$. Given an input list L with original rank map ρ, we take as initial geometric state its rank word

$$x(0) = x_0 = (\rho(1), \rho(2), \ldots, \rho(n)) \in \mathcal{P}_n, \tag{25}$$

so that the relaxation carries this fixed geometric representative toward the sorted vertex v_s.

4.2 Main Theorem and Proof

Theorem 4.1 (Ambient Euclidean Contraction)**.** *Let $x(0) \in \mathbb{R}^n$ be the initial rank word corresponding to an unsorted input list and let $v_s = (1, 2, \ldots, n)$ be the sorted vertex of the permutohedron $\mathcal{P}_n$. Define*

$$D_0 = \|x(0) - v_s\|^2. \tag{26}$$

Under the gradient flow

$$\dot{x}(t) = -\nabla V(x(t)) = v_s - x(t), \tag{27}$$

the following statements hold:

1. *The solution satisfies*

$$x(t) = v_s + \big(x(0) - v_s\big)e^{-t}, \tag{28}$$

$$\|x(t) - v_s\|^2 = \|x(0) - v_s\|^2 e^{-2t}. \tag{29}$$

2. *For a threshold $0 < \epsilon < \sqrt{D_0}$, the condition $\|x(t) - v_s\| \leq \epsilon$ holds once*

$$t \geq \frac{1}{2}\ln\Big(\frac{D_0}{\epsilon^2}\Big). \tag{30}$$

3. *For the reverse rank word* $x(0) = (n, n-1, \ldots, 1)$,

$$D_0 = \sum_{i=1}^{n} (n + 1 - 2i)^2 = \frac{n(n^2-1)}{3} = \Theta(n^3), \tag{31}$$

and therefore

$$V(x(0)) = \frac{D_0}{2} = \frac{n(n^2-1)}{6}. \tag{32}$$

For fixed $\epsilon = \Theta(1)$*, the threshold time is* $t = \Theta(\ln n)$.

Proof. We begin by deriving the gradient flow dynamics directly from the potential V. A straightforward computation shows that

$$\nabla V(x) = \big(x_1 - 1,\ x_2 - 2,\ \ldots,\ x_n - n\big). \tag{33}$$

Hence, the gradient flow is given by

$$\dot{x}(t) = -\nabla V(x(t)) = \big(1 - x_1(t),\ 2 - x_2(t),\ \ldots,\ n - x_n(t)\big). \tag{34}$$

Because the system decouples, each coordinate x_i satisfies the first-order linear differential equation

$$\dot{x}_i(t) = i - x_i(t), \tag{35}$$

with initial condition $x_i(0)$. Its solution is

$$x_i(t) = i + \big(x_i(0) - i\big)e^{-t}. \tag{36}$$

Collecting these coordinate solutions, we obtain

$$x(t) = v_s + \big(x(0) - v_s\big)e^{-t}, \tag{37}$$

so that

$$\|x(t) - v_s\|^2 = \|x(0) - v_s\|^2 e^{-2t}. \tag{38}$$

This establishes statement 1.

For statement 2, define the *initial disorder* $D_0 = \|x(0) - v_s\|^2$. The system is effectively sorted when

$$\|x(t) - v_s\|^2 = D_0 e^{-2t} \le \epsilon^2. \tag{39}$$

Taking logarithms yields

$$t \ge \frac{1}{2} \ln\Big(\frac{D_0}{\epsilon^2}\Big). \tag{40}$$

Reverse-permutation radius. If $x(0) = (n, n-1, \ldots, 1)$ then $x_i(0) - i = n + 1 - 2i$. The standard centered-square identity gives

$$D_0 = \sum_{i=1}^{n} \big(n + 1 - 2i\big)^2 = \frac{n(n^2-1)}{3} = \Theta(n^3). \tag{41}$$

Consequently the initial potential at the reverse rank word is

$$V(x(0)) = \frac{1}{2} D_0 = \frac{n(n^2-1)}{6}. \tag{42}$$

Hence $t = \Theta(\ln n)$. □

4.3 Normalized Comparison Clock

To place the informational and metric processes on a common asymptotic scale without identifying them, observe that comparison sorting must resolve total order-type entropy $\log_2(n!)$, whose leading scale is $n \log n$. Dividing by n gives the information per item and motivates the normalized comparison clock

$$s_k = \frac{k}{n}. \tag{43}$$

A binary comparison contributes at most one bit in total, hence at most $1/n$ bit per item. This is only a scale comparison; it asserts no coupling between an individual outcome and a fixed increment of the Euclidean trajectory.

Proposition 4.2 (Normalized comparison time)**.** *The decision-tree lower bound gives $s_k = \Omega(\log n)$, while optimal comparison sorting supplies the matching $s_k = O(\log n)$ upper bound. Hence the normalized optimal comparison scale is*

$$s_k = \Theta(\log n). \tag{44}$$

Thus the continuous threshold time $t = \Theta(\ln n)$ in Theorem 4.1 lies on the same macroscopic scale as optimal comparison sorting.

Proof. The decision-tree lower bound gives

$$k \geq \log_2(n!) = n \log_2 n - O(n), \tag{45}$$

while optimal comparison sorts achieve $k = O(n \log n)$. Dividing the comparison count by n gives the respective bounds $s_k = \Omega(\log n)$ and $s_k = O(\log n)$, and together they give $s_k = \Theta(\log n)$. □

This normalization does not provide another proof of the classical lower bound. That rigorous argument remains the decision-tree proof of Section 2. The continuous model simply supplies a compatible geometric scale once comparison information is measured per element.

4.4 Constraint Geometry and the Ambient Flow

The exact flow in Theorem 4.1 is an ambient Euclidean benchmark. It is already compatible with the convex geometry of the permutohedron: $\mathcal{P}_n$ lies in the affine hyperplane

$$\sum_{i=1}^{n} x_i = \frac{n(n+1)}{2}, \tag{46}$$

and both $x(0)$ and v_s lie in $\mathcal{P}_n$. Since $\mathcal{P}_n$ is convex, the straight segment

$$x(t) = v_s + \big(x(0) - v_s\big)e^{-t} \tag{47}$$

remains in $\mathcal{P}_n$ for all $t \geq 0$.

It is important not to identify comparison hyperplanes with the boundary facets of $\mathcal{P}_n$. The hyperplanes

$$x_i = x_j \tag{48}$$

are braid-arrangement walls in the ambient affine space; they slice $\mathcal{P}_n$, but they are not, in general, facets of the permutohedron. The facets of the standard permutohedron are instead described by subset-sum inequalities of the form

$$\sum_{i \in S} x_i \geq \frac{|S|(|S|+1)}{2}, \qquad \varnothing \neq S \subsetneq \{1, \ldots, n\}, \tag{49}$$

together with the fixed total-sum equality. Thus projection is not forced merely by the fact that the trajectory lies in $\mathcal{P}_n$.

The distinction now becomes concrete. Comparison-feasible sets describe information, whereas constraints on a metric trajectory would restrict motion. If a closed convex constraint still contains the target, convexity gives a direct answer.

Proposition 4.3 (Projection is inactive when the target is feasible)**.** *Let $K \subseteq \mathcal{P}_n$ be a closed convex set containing v_s. For every $x \in K$,*

$$v_s - x \in T_K(x). \tag{50}$$

Here $T_K(x)$ is the tangent cone of K at x. Therefore its Euclidean projection satisfies

$$\Pi_{T_K(x)}(v_s - x) = v_s - x, \tag{51}$$

and the projected flow agrees with the ambient flow:

$$\dot{x} = v_s - x. \tag{52}$$

Proof. Since $x, v_s \in K$ and K is convex,

$$x + \lambda(v_s - x) \in K \qquad (0 \le \lambda \le 1). \tag{53}$$

Thus $v_s - x$ is a feasible tangent direction at x, so projecting it onto the tangent cone leaves it unchanged. □

If a convex feasible region contains v_s, projection is therefore inactive. If instead the region excludes v_s, constrained descent approaches the unique point in the region nearest to v_s, not v_s itself. Comparison half-spaces should consequently be read as a description of information reduction, while the Euclidean flow is a separate ambient contraction benchmark. Comparisons remove informational ambiguity; gradient flow removes metric distance.

4.5 Remarks

Theorem 4.1 demonstrates that continuous gradient descent under the potential $V(x) = \frac{1}{2}\|x - v_s\|^2$ gives exact exponential contraction of ambient rank displacement. Proposition 4.2 then compares this motion with a normalized comparison clock: by definition one comparison increments that clock by $1/n$, and the classical optimal comparison scale becomes $\Theta(\log n)$. The decision-tree proof remains the rigorous lower-bound argument; the flow supplies a compatible geometric benchmark. More broadly, a descent formulation can clarify a process by identifying an energy that decreases along its path. In this respect the viewpoint echoes the insight of Jordan, Kinderlehrer, and Otto, who reinterpreted the Fokker–Planck equation as a gradient flow minimizing a free energy functional under the Wasserstein metric [9].

Figure 4 brings the discrete and continuous geometries together. Vertices represent rank words, and the gradient flow traces a smooth ambient trajectory inside the convex permutohedron toward the sorted vertex. The left panel contrasts this trajectory with the adjacent–swap path: local algorithms advance by short moves along edges, while the ambient relaxation cuts across the polytope by straight-line descent. This path is not a graph-geodesic on the 1-skeleton and is not the literal trajectory of a discrete sorting algorithm.

The right panel presents the same motion in scalar form. The distance to the sorted state contracts exponentially as $r(t) = \|x(t) - v_s\|_2$ decreases. Viewed in (r, t, V)-space, the trajectory resembles motion through a curved landscape: the system rolls downhill under the potential, like a particle descending a gravitational well toward equilibrium. In both panels the example is the same list $L = [5, 4, 2]$ used earlier:

the rank word begins at $(3, 2, 1)$ and flows toward $(1, 2, 3)$, corresponding to the sorted output $[2, 4, 5]$. The initial radius is $r_0 = \|(3, 2, 1) - (1, 2, 3)\|_2 = \sqrt{8}$, and the initial dissipation is $dV/dt = -\|\nabla V(x(0))\|_2^2 = -8$.

Together, the panels keep the two roles visible: the left contrasts discrete and continuous paths, while the right isolates exponential metric contraction. The classical $\Theta(n \log n)$ scale is therefore consistent with logarithmic contraction on the normalized comparison clock.

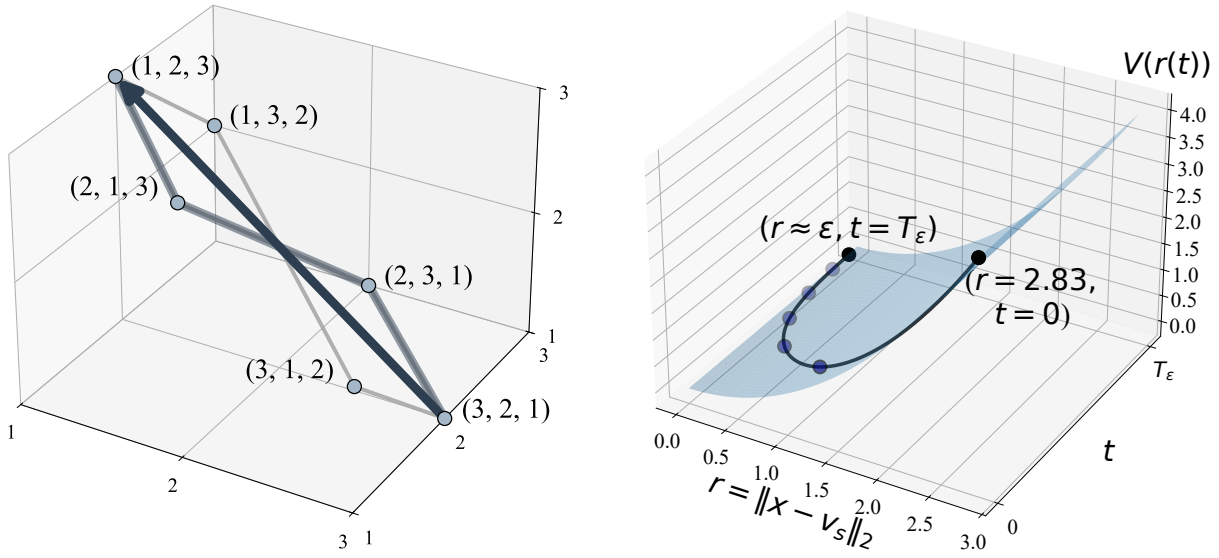

Figure 4: Gradient-flow relaxation under $V(x) = \frac{1}{2}\|x - v_s\|_2^2$. **Left:** Straight-line ambient descent on $\mathcal{P}_3$ from $(3, 2, 1)$ to $(1, 2, 3)$, contrasted with the adjacent-swap path. **Right:** Exponential contraction of $r(t) = \|x(t) - v_s\|_2$ from $r_0 = \sqrt{8}$ to a threshold ε. This is a Euclidean benchmark, not the literal path of a discrete sorting algorithm; under normalized comparison time, its $\Theta(\log n)$ contraction scale matches the $\Theta(n \log n)$ comparison scale.

5 Conclusion

The central challenge in efficient sorting is overcoming the factorial scale of the permutation space. Established algorithms do this by exploiting structure rather than by searching faster. MergeSort uses the recurrence

$$T(n) = 2T(n/2) + O(n) \tag{54}$$

to obtain an $O(n \log n)$ running time through divide-and-conquer, and von Neumann's implementation showed how recursive design could yield both theoretical and practical efficiency [10]. QuickSort uses pivot choices to prune the search space in expectation; HeapSort imposes heap structure. In each case, the algorithm succeeds by organizing the permutation space before it is exhausted.

From a geometric perspective, the gain over local sorting appears as a contrast between motion and information. Adjacent-swap algorithms traverse the permutohedron along its edges, resolving one inversion at a time and producing the classical $\Theta(n^2)$ behavior. Arbitrary comparisons act differently: they impose constraints that reshape the feasible region as a whole. This is the geometric meaning of the decision-tree improvement. Sorting succeeds not by walking the polytope faster, but by collapsing the feasible order-type set. This theme is consistent with structured decomposition methods in combinatorial algorithms [11] and

with recent enumerative work of Harris, Kretschmann, and Martínez Mori, where QuickSort comparisons are counted by parking preference lists with $n-1$ "lucky cars" [6].

The continuous model adds a third view: ambient gradient flow on the permutohedron. This relaxation is not the literal path of a sorting algorithm, but it clarifies the geometries at play. Adjacent swaps give the graph geometry of the 1-skeleton. Arbitrary comparisons give an information geometry on candidate rank maps. The Euclidean flow gives a smooth rank-displacement benchmark inside the same polytope. As the flow contracts the ambient distance to the sorted order according to

$$\|x(t) - v_s\|^2 = \|x(0) - v_s\|^2 e^{-2t}, \tag{55}$$

time becomes a measure of geometric convergence. Discrete complexity re-enters through comparison information: one binary comparison advances the normalized clock by $1/n$. The decision-tree lower bound gives $\Omega(\log n)$ normalized units, and optimal comparison sorting supplies the matching $O(\log n)$ upper bound.

Shannon's information-theoretic analysis made clear that search is governed by available information [4]. Complexity theory develops this principle across many computational settings [12]; Mann's perspective on search emphasizes the way constraints organize rather than merely restrict a space [13]. The present model gives this idea a concrete geometric form for sorting. Factorial search collapses into log-linear time when comparisons supply enough global information to organize the candidate rank maps. In that sense, local motion and global structure meet inside the same object: the permutohedron, its comparison half-spaces, and the ambient flow toward order. The analogy is therefore not that comparisons literally implement the quadratic flow, but that informational refinement and metric contraction reveal parallel geometric views of sorting on a common logarithmic scale.

Acknowledgment

The author used software tools, including AI assistance, for figure preparation, algebraic checks, and editing. The author is responsible for all mathematical content and conclusions.

Disclosure of interest. The author reports no conflict of interest.

Funding. No funding was received for this research.

References

[1] Knuth DE. Big Omicron and Big Omega and Big Theta. SIGACT News. 1976;8(2):18-24.

[2] Cormen TH, Leiserson CE, Rivest RL, Stein C. Introduction to Algorithms. 4th ed. Cambridge, MA: MIT Press; 2022.

[3] Knuth DE. The art of computer programming, volume 3: sorting and searching. Reading, MA: Addison-Wesley; 1973.

[4] Shannon CE. A mathematical theory of communication. Bell Syst Tech J. 1948;27:379-423, 623-56. Parts I–II; part I doi: 10.1002/j.1538-7305.1948.tb01338.x; part II doi: 10.1002/j.1538-7305.1948.tb00917.x.

[5] Blelloch GE, Dobson M. The geometry of tree-based sorting. In: Proceedings of the 50th International Colloquium on Automata, Languages, and Programming (ICALP 2023). vol. 261 of LIPIcs. Paderborn, Germany: Schloss Dagstuhl–Leibniz-Zentrum für Informatik; 2023. p. 26:1-26:19. Available from: https://doi.org/10.4230/LIPIcs.ICALP.2023.26.

[6] Harris PE, Kretschmann J, Martínez Mori JC. “Lucky cars” and the quicksort algorithm. The American Mathematical Monthly. 2024;131(5):417-23.

[7] Ziegler GM. Lectures on polytopes. New York, NY: Springer-Verlag; 1995.

[8] Lee E, Mitchell C, Vindas-Meléndez AR. Stack-sorting simplices: geometry and lattice-point enumeration. arXiv preprint. 2023. ArXiv:2308.16457.

[9] Jordan R, Kinderlehrer D, Otto F. The variational formulation of the Fokker–Planck equation. SIAM J Math Anal. 1998;29(1):1-17.

[10] von Neumann J. First draft of a report on the EDVAC. Philadelphia, PA: Moore School of Electrical Engineering, University of Pennsylvania; 1945.

[11] Hopcroft JE, Tarjan RE. Algorithm 447: efficient algorithms for graph manipulation. Commun ACM. 1973;16(6):372-8.

[12] Arora S, Barak B. Computational complexity: a modern approach. Cambridge University Press; 2009.

[13] Mann Z. The top eight misconceptions about NP-hardness. Computer. 2017;50(5):72-9.